A Guide to Reproducing: An Oracle-based Approach for Price-setting Problems in Logistics

Amine Bahij
amine.bahij@tu-dortmund.de

July 24, 2026

Contents

1 Introduction

This guide explains how to reproduce the computational experiments from the paper *"An Oracle-based Approach for Price-setting Problems in Logistics"* (Pommerening, Hügging, Henke, Buchheim, Clausen, 2026).

The reference implementation is the **Oracle Paper** Python package, built on top of **BilevelPy**, a reusable framework for bilevel optimization.

The implementation is available in the Oracle Paper repository. The generated project documentation is published at the Oracle Paper documentation website. BilevelPy is available through its GitHub repository and documentation website.

The package provides four solution approaches for the Price-Setting Bilevel Hub Location Problem (PS-BHLP):

Model	Paper Name	Method
`ps_hlp`	PS-HLP	Single-level Big-M reformulation
`ppc_hlp`	PPC-HLP	Lagrangian decomposition with precedence constraints
`pc_hlp`	PC-HLP	Lagrangian decomposition without precedence constraints
`ps_bhlp`	PS-BHLP	Bilevel formulation via Julia / BilevelJuMP

2 Installation

2.1 Prerequisites

- **Gurobi 11+** with a valid license. See Gurobi Academic Program if you are eligible.
- **Python 3.11–3.14**
- **Poetry** (recommended) or pip
- **Julia 1.10+** — only required for the `ps_bhlp` model

2.2 Installation Steps

1. Clone the repository and enter the project directory:

```
git clone https://github.com/Institute-of-
   Transport-Logistics/oracle-price-settings-
   logistics.git
cd oracle-price-settings-logistics
```

2. Install with Poetry:

```
poetry install
```

3. Verify the Python installation:

```
poetry run python -c "from␣oracle_paper.models.
   ps_hlp␣import␣PS_HLP;␣print('Installation␣
   successful')"
```

4. (Optional) Set up Julia for the PS-BHLP model:

```
poetry run setup-julia
```

   This installs the required Julia packages (JuMP, Gurobi, BilevelJuMP, JSON). If Julia is not found, the script prints a warning — the other three models work without Julia.

For more details, see the online installation guide or the installation guide in the repository.

3 Quick Example

The following script builds a minimal dataset, solves it with the PS-HLP (Big-M) model, and prints the solution:

```python
from bilevelpy.core.datasets import Dataset
from bilevelpy.data.builder import DatasetBuilder
from bilevelpy.data.loaders import HLPLoader
from bilevelpy.data.processor import HLPNodeSelector,
    HLPCostScaling
from bilevelpy.solver import ModelSolver


from oracle_paper.data.generator.
    bilevel_client_generator import
    BilevelClientGenerator
from oracle_paper.data.processor.client_ranker import
    LinearClientRanker
from oracle_paper.models.ps_hlp import PS_HLP

dataset = (
    DatasetBuilder()
    .pipe(HLPLoader(Dataset.CAB100))
    .pipe(HLPNodeSelector(n_nodes=10))
    .pipe(HLPCostScaling(scaling_factor=100))
    .pipe(BilevelClientGenerator(clients_per_route=5))
    .pipe(LinearClientRanker())
    .build()
)

model = PS_HLP(n_hubs=2, alpha=0.5, data=dataset)
solution = ModelSolver(model).solve()
print(solution)
```

Additional examples for all four models can be found in the src/oracle_paper/examples/ directory.

4 Reproducing the Paper's Benchmarks

4.1 Running the Benchmarking Tool

The paper's computational results are produced by the Streamlit benchmarking UI. Launch it with:

```
poetry run benchmark
```

This opens the interface in your browser.

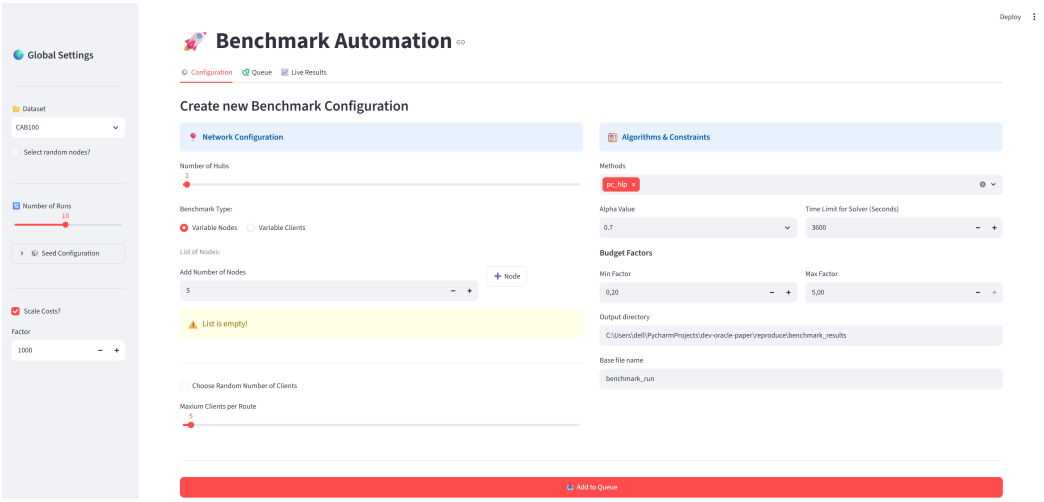

Figure 1: Benchmarking Tool — main interface

4.2 Global Settings (Sidebar)

The sidebar controls the dataset, randomness, number of runs, seeds, and cost scaling.

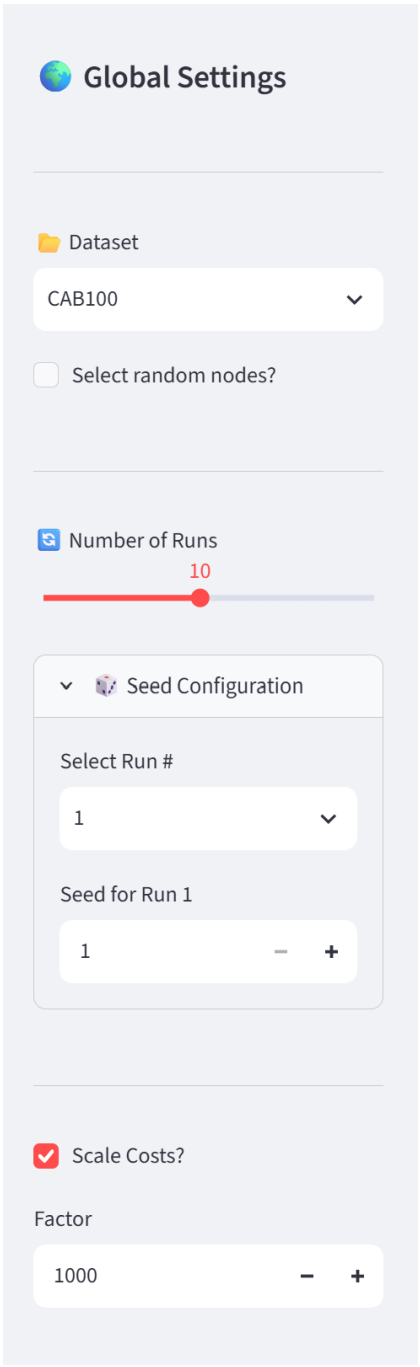

Figure 2: Sidebar with global settings

Dataset Select the dataset. The paper uses `CAB100` for all experiments.

Number of Runs How many times each configuration is repeated with different random seeds. The paper uses 10 runs per configuration.

Seed Configuration Expand this section to set individual seeds per run. By default, run i gets seed i.

Scale Costs When enabled, transport costs c_{ij} are divided by the given factor. The paper uses a scaling factor of 1000.

4.3 Configuration Tab

This tab is where you define a benchmark job and add it to the queue.

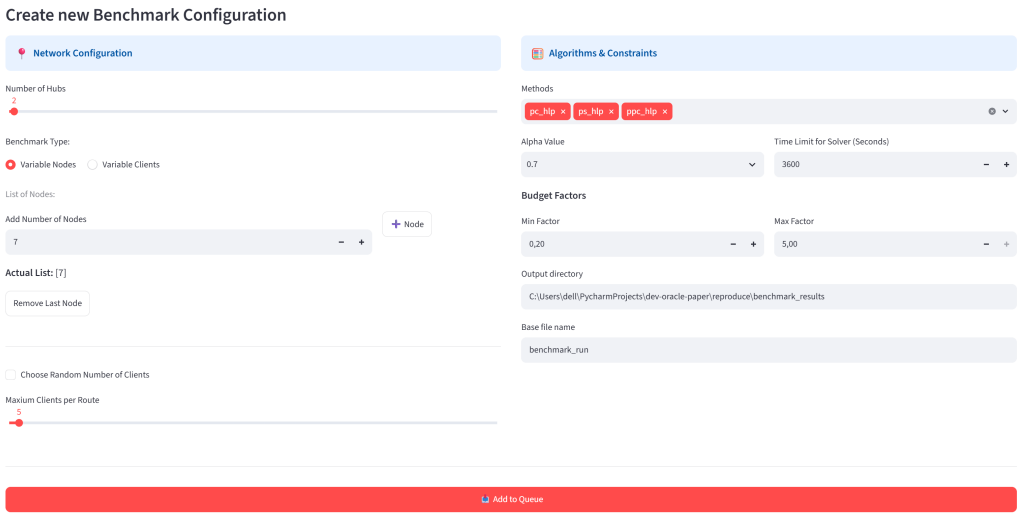

Figure 3: Configuration tab

Network Configuration (left column)

- **Number of Hubs** (κ): the paper uses $\kappa = 3$.
- **Benchmark Type**: choose `Variable Nodes` (vary $|V|$, fix γ) or `Variable Clients` (fix $|V|$, vary γ).

- For `Variable Nodes`: add node counts to the list (e.g., 5, 10, 15, 20). Also set the fixed number of clients per route (the paper uses $\gamma = 5$).
- For `Variable Clients`: set the fixed number of nodes, then add client counts to the list.

Algorithms & Constraints (right column)

- **Methods**: select which models to benchmark. The paper compares `ps_hlp`, `ppc_hlp`, and `pc_hlp`.
- **Alpha Value**: inter-hub discount factor. The paper uses $\alpha \in \{0.5, 0.7\}$.
- **Time Limit**: solver time limit in seconds. The paper uses 3600 (1 hour).
- **Budget Factors**: the paper uses min factor 0.2 and max factor 5.0.
- **Output directory**: where results are saved. Defaults to `reproduce/benchmark_results/`.
- **Base file name**: prefix for CSV output files.

Click **Add to Queue** to save the configuration. Repeat for each parameter combination you want to benchmark.

4.4 Queue Tab

Shows all queued benchmark jobs.

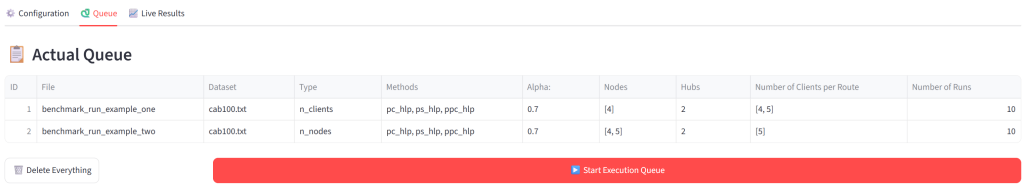

Figure 4: Queue tab with pending jobs

Click **Start Execution Queue** to run all jobs sequentially. Each job sweeps over the configured parameter ranges and runs each combination for the specified number of runs.

4.5 Results Tab

Live monitoring of the running benchmark.

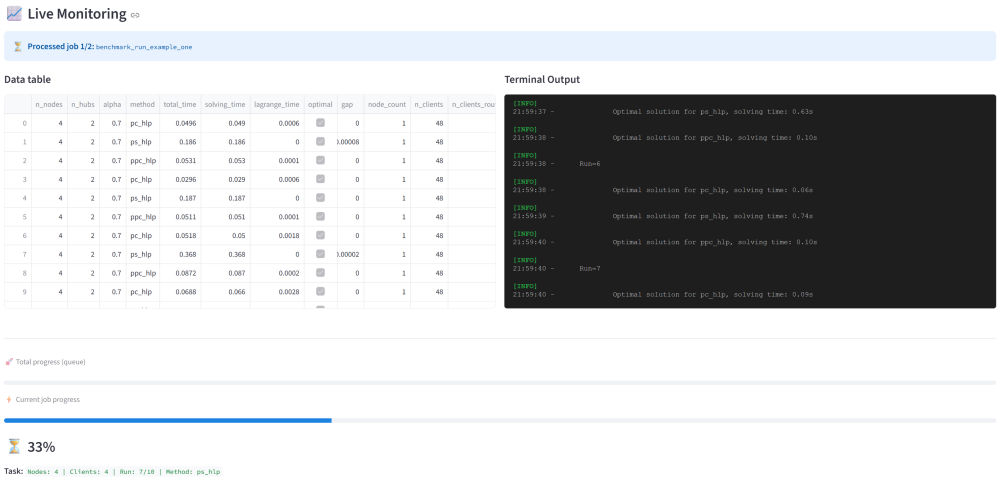

Figure 5: Results tab during execution

The left panel shows the results table updating in real time. The right panel shows solver log output. A progress bar tracks the current job and the overall queue.

5 Output Structure

Benchmark results are saved in the configured output directory (default: `reproduce/benchmark_results/`). The folder structure is:

```
<output_dir>/
  YYYY_MM_DD_HH_MM/                          # Timestamp of
     benchmark run
    n_nodes/                                  # If varying
       nodes
      node_range_<start>_<end>/
        clients_<n_clients>/
          n_hubs_<n_hubs>/
            <file_name>_raw.csv
            execution.log
    n_clients/                                # If varying
       clients
      n_clients_range_<start>_<end>/
        n_nodes_<n_nodes>/
          n_hubs_<n_hubs>/
            <file_name>_raw.csv
            execution.log
```

5.1 CSV Columns

Each `_raw.csv` file contains:

- `n_nodes`: number of nodes, $|V|$
- `n_hubs`: number of hubs, κ
- `alpha`: inter-hub discount factor
- `method`: model identifier (`ps_hlp`, `pc_hlp`, `ppc_hlp`, `ps_bhlp`)
- `total_time`: total time in seconds (solving + Lagrange)
- `solving_time`: Gurobi solving time in seconds
- `lagrange_time`: Lagrange multiplier computation time
- `optimal`: whether the solution is provably optimal

- `gap`: MIP gap if not optimal
- `node_count`: branch-and-bound nodes explored
- `n_clients`: total number of clients in the instance
- `n_clients_route`: clients per route, γ
- `random_n_clients_route`: whether route client counts vary
- `min_budget_factor`, `max_budget_factor`
- `run_number`: run index
- `seed`: random seed
- `sanctioned`: whether the method was skipped (after repeated non-optimal results)

6 Pre-packaged Results

The `reproduce/benchmark_results/` directory contains the published CSV result files. These can be loaded directly for analysis without re-running the benchmarks.

7 Paper-to-Code Reference

Paper	**Method Name**	**Code**
PS-HLP	Big-M reformulation	`ps_hlp`
PPC-HLP	Lagrange with precedence	`ppc_hlp`
PC-HLP	Lagrange without precedence	`pc_hlp`
PS-BHLP	Bilevel (Julia)	`ps_bhlp`

Table 1: Paper-to-code naming convention

8 Documentation

The full documentation is published at https://institute-of-transport-logistics.github.io/oracle-price-settings-logistics/. The Markdown sources are available in the `docs/` directory. To build and serve the documentation locally using MkDocs:

```
poetry run mkdocs serve
```

Then open `http://localhost:8000` in your browser.

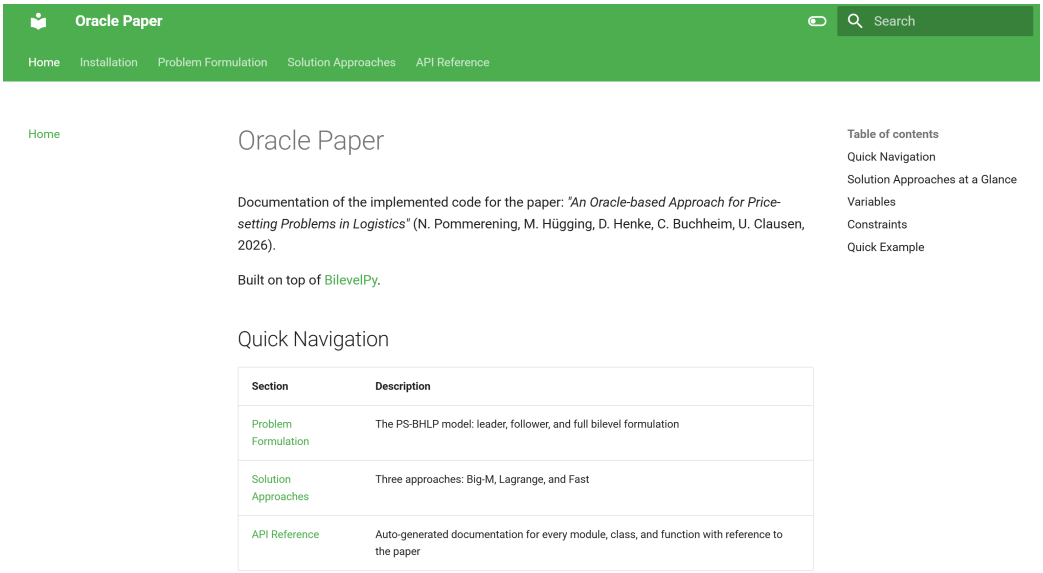

Figure 6: Documentation site index page